RetroNNetScaler: Contour Upscaling for Hard-Palette NES Pixel Art

RetroNNetScaler Project
Technical Report
https://github.com/amirsdream/RetroNNetScaler
August 2026

***Abstract*—Upscaling palette-limited Nintendo Entertainment System (NES) frames for modern 4K displays differs fundamentally from natural-image super-resolution. Continuous filters and photograph-oriented networks invent mid-tone "cream" colours and silky outlines that erase the aliased aesthetic of 8-bit games. We present RetroNNetScaler, a Windows realtime pipeline that embeds a NES core (SimpleNES), presents frames through Direct3D 11, and offers three hard-palette production modes: *Clarity* (maximum integer nearest-neighbour), *Smooth* (xBR-style tip wedges with neighbourhood colour snap), and *Contour* (Kopf-style planarized diagonals with *shared global half-planes* so that long 45° stairs become a single continuous edge). We further document a TinySR neighbour-classification CNN (≈ 52k weights) trained against a Scale2x-family discrete teacher: palette drift can be forced to zero by construction, yet unconstrained decoding produces multicolour edge fringes. The central methodological result is that one should discretise the output domain first; for continuous diagonals without cream, adjacent cells must agree on one geometric edge equation rather than regress RGB. Contour requires no learned weights at runtime and maps naturally to an FPGA streaming datapath.**

***Index Terms*—pixel-art upscaling, NES, Contour, Kopf–Lischinski, xBR, hard palette, realtime D3D11, discrete super-resolution, FPGA.**

I. Introduction

Classic console graphics were authored on a square pixel lattice with a small discrete palette. When those frames are shown on a 3840×2160 panel, naive bilinear or bicubic filters blur edges and invent colours that never existed in the ROM. Deep super-resolution models trained for photographs or anime with continuous losses tend to produce a *silky* look: soft outlines, chroma bleed, and palette drift. For retro enthusiasts and preservationists, that look is a regression rather than an improvement.

Rule-based pixel-art scalers such as EPX/Scale2x and later pattern-based methods (e.g., hqx, xBRZ) address diagonals by rewriting blocks from existing neighbour colours rather than photograph-style interpolation [1]–[4]. Kopf and Lischinski depixelization goes further toward resolution-independent vectors [5], which many players perceive as too smooth for classic NES draw. Our product goal is narrower: hard colours (no mid-tone cream); cleaner, continuous 45° diagonals; realtime 4K presentation; and a path toward FPGA implementation.

This paper documents the RetroNNetScaler prototype: system architecture (Section III), Clarity / Smooth / Contour algorithms (Section IV), the shared half-plane insight that fixes EPX zigzags (Section V), implementation notes (Section VI), the TinySR CNN negative result (Section VII), discussion (Section VIII), and an FPGA-oriented roadmap (Section IX).

II. Related Work

A. Classical Pixel-Art Scalers

EPX expands each source pixel into a 2×2 block using equality tests on cardinal neighbours and never invents colours; the rule set was published in the practitioner literature by Thomas [1] (crediting Eric Johnston, ca. 1992). Scale2x implements the same class of logic for AdvanceMAME [2]. Stasik and Balcerek [3] analyse later pattern-based scalers (including hqx and xBRZ) and note their restriction to fixed integer factors. Seo *et al.* [4] likewise survey Scale2x/xBRZ and emphasise preserving discrete palette and grid structure—the constraint we enforce throughout RetroNNetScaler.

B. Depixelization and Continuous SR

Kopf and Lischinski [5] build a similarity graph, planarize crossing diagonals with curves / sparse / islands heuristics, reshape cells via a generalized Voronoi diagram on nodes and half-edges, then fit contour splines. Kreuzer *et al.* [6] present a GPU implementation aimed at realtime emulator rates (I3D 2015 poster). Full spline fitting yields the classic "inked" vector look; we adopt the *planarization + hard ownership* stage and deliberately omit soft B-spline shading so fills stay flat NES colours.

Photograph-oriented SR—including ESRGAN [7], Real-ESRGAN [8], and SwinIR [9]— optimises continuous or perceptual losses. As discussed by Seo *et al.* [4], such models can smooth structure or infer colours outside a discrete palette. ESPCN-style networks [10] remain attractive for FPGA targets because of small parameter counts; we adopt that footprint in Section VII but replace RGB residual regression with a 9-way neighbour classifier.

III. System Architecture

The PC prototype is organised as a single process:

```
NES core (SimpleNES)
     |  256 x 240 RGBA @ ~60.098 Hz
     v
D3D11 upload -> scaler CS -> integer letterbox present
     |
Modes: Clarity | Smooth | Contour | NIS | Nearest | Bilinear
```

TABLE I
SCALING MODES IN THE CURRENT BUILD.

Mode	Algorithm	Palette
Clarity	Max integer nearest	Exact LQ
Smooth	xBR-sharp + snap	Hard 3×3
Contour	Shared half-planes	Exact neighbour
NIS	NVIDIA Image Scaling	Continuous
Nearest / Bilinear	Baselines	Exact / blend

A custom Win32 launcher provides ROM selection (browse / drag-and-drop), mode chips (Clarity / Smooth / Contour), and output resolution (1080p / 1440p / 4K). Preferences persist under AppData. Gameplay is paced to the NES NTSC rate independent of panel refresh. Presentation uses the maximum integer scale factor that fits the backbuffer, with phase-locked letterboxing so that the presented content size is exactly $w_{\mathrm{src}}s \times h_{\mathrm{src}}s$ and output pixels remain uniform width.

IV. Methods

A. Clarity: Integer Oracle

Clarity selects

$$s = \min\big(\lfloor W/w_{\mathrm{src}} \rfloor,\ \lfloor H/h_{\mathrm{src}} \rfloor\big), \tag{1}$$

point-samples NES texels into a canvas of size $(w_{\mathrm{src}}\, s) \times (h_{\mathrm{src}}\, s)$, and letterboxes into the backbuffer. At 1080p, NES 256×240 typically yields s=4 (1024×960); at 4K, s=9 (2304×2160). Clarity is the colour-perfect reference: blocky, but any subsequent filter must improve diagonals without introducing cream.

B. Smooth: Sharp xBR with Hard Snap

Smooth implements an xBR-inspired sharp pass in HLSL (edge-directed tip geometry in the spirit of the pattern-based family surveyed by [3]). For each output sample we evaluate edge dominance with a YUV colour distance, restrict writes to outer tip wedges of each quadrant, apply an outline guard on high-contrast corners, and finally apply `snapExact9`: the result is replaced by the nearest of the nine low-resolution (LQ) neighbourhood colours $\{A, \ldots, I\}$. Mid-tones cannot survive the snap, which eliminates cream while keeping intentional tip geometry.

C. Contour: Shared Half-Plane Diagonals

Contour is the production answer to "sharp fills, continuous edges." For each output sample inside LQ cell E at integer coordinates (s_x, s_y) with fractional offsets $(f_x, f_y) \in [0, 1)^2$, let

$$\mathbf{p} = (s_x + f_x,\ s_y + f_y), \quad U = p_x + p_y, \quad V = p_x - p_y. \tag{2}$$

The algorithm proceeds as follows:

1) Fetch the 3×3 neighbourhood $A \ldots I$ and test similarity with Kopf/hqx YUV thresholds [5]: $|\Delta Y| \le 48/255$, $|\Delta U| \le 7/255$, $|\Delta V| \le 6/255$.

Algorithm 1 Planar diagonal selection in a 2×2 block.

Require: colours a, b, d, e of $\left[\begin{smallmatrix} a & b \\ d & e \end{smallmatrix}\right]$
Ensure: 0 (none), 1 (keep a–e), or 2 (keep b–d)
$ae \leftarrow \mathrm{Similar}(a, e)$; $bd \leftarrow \mathrm{Similar}(b, d)$
if $\neg ae \wedge \neg bd$ **then**
  return 0
end if
if all four cardinal edges similar **then**
  return 0 {fully connected shade}
end if
if $ae \wedge \neg bd$ **then**
  return 1
end if
if $bd \wedge \neg ae$ **then**
  return 2
end if
if any cardinal similar **then**
  return 0 {drop crossing diagonals}
end if
score both diagonals (curves / sparse / islands)
return higher-scoring diagonal, or 0 on tie

2) Planarize each of the four 2×2 blocks around E (Algorithm 1): keep at most one diagonal (A–E or B–D, etc.), using a local curves / sparse / islands score when both diagonals cross.
3) Assign colour by a **global** half-plane on U (slash /) or V (backslash \), with bisectors at half-integers so the dark cell and the light cell of a stair share the *same* edge equation (Section V).
4) If no 45° rule fires, apply 2:1 / 1:2 hard wedges (still exact neighbour colours).

The output colour is always one of $A \ldots I$—never a blend. No network weights are required at runtime.

V. Why Local EPX Cuts Still Look Pixelated

Classic EPX paints a triangle inside each cell, e.g., $f_x + f_y < \frac{1}{2}$ for a top-left cut. On a long slash stair, the dark cell's cut and the light cell's cut often sit on **parallel but offset** lines—different constants of $U = x + y$. The visible border becomes a thick zigzag at the scale of *source* pixels: still "pixelated," even though each cut is continuous inside its cell.

Contour forces both sides onto one shared bisector. With $U_s = s_x + s_y$ and $V_s = s_x - s_y$:

$$\text{TL keep } B\text{–}D: \quad U < U_s + \tfrac{1}{2} \ \Rightarrow\ B, \tag{3}$$

$$\text{BR keep } F\text{–}H: \quad U > U_s + \tfrac{3}{2} \ \Rightarrow\ F, \tag{4}$$

$$\text{TR keep } B\text{–}F: \quad V > V_s + \tfrac{1}{2} \ \Rightarrow\ B, \tag{5}$$

$$\text{BL keep } D\text{–}H: \quad V < V_s - \tfrac{1}{2} \ \Rightarrow\ D. \tag{6}$$

Complementary corners of a stair therefore agree on a single straight 45° interface (e.g., $U = U_s + \frac{1}{2}$ meeting $U = U_s + \frac{3}{2}$ from the adjacent cell), recovering continuity without blending.

TABLE II
CLASSIFIER VALIDATION ON SUPER MARIO BROS. HOLD-OUT FRAMES (ILLUSTRATIVE MEANS).

Metric	NN	CNN	Teacher
Acc. (all)	—	$\approx 96\%$	100%
Acc. (edge)	—	$\approx 72\%$	100%
Palette drift	0	0	0

VI. IMPLEMENTATION NOTES

Clarity, Smooth, and Contour share one compute shader (`shaders/tiny_sr.hlsl`), selected by a mode constant. Contour dispatches at destination resolution with integer scale s from the present path. The shader is compiled at runtime with `D3DCompile` (`cs_5_0`). The Win32 launcher maps the Contour chip to the internal `ScaleMode::Neural` enum for preferences compatibility; the CLI accepts `--scale neural`.

Integer phase lock uses content size exactly $w_{\mathrm{src}}s \times h_{\mathrm{src}}s$, an integer-centred viewport, point sampling after the scaler, and optional one-pixel phase nudges.

VII. THE NEURAL EXPERIMENT (NEGATIVE RESULT)

Before Contour, we trained a TinySR network to predict, for each LQ pixel, a class in $\{A, \ldots, I\}$ for every subpixel of a 4×4 block (pixel-shuffle head). Decoded colour is exactly $\mathrm{neighbour}[\arg\max]$—zero palette drift by construction.

```
Conv3x3 C=32 ReLU -> Conv3x3 C=32 ReLU
  -> Conv3x3 9*4*4 -> pixel_shuffle(r=4)
  -> logits (9,H*4,W*4)   ~51.8k params
```

Labels came from a Scale2x-family geometric teacher [2] chained to $4\times$ (exact neighbour indices; no blended colours). Non-centre classes are rare (≈ 0.3–1% of pixels), so training used class-weighted cross-entropy with a large edge multiplier, edge-biased crops, AdamW, and early stopping on edge-class accuracy (CUDA, GeForce RTX 4080).

A. Failure Modes

a) Cream.: Residual RGB regression underfits sparse EPX/Scale2x deltas; raising gain without discrete decoding invents mid-tones.

b) Fringes.: At roughly 72% edge accuracy, false non-E classes copy a wrong palette entry (e.g., HUD red into a black silhouette), appearing as multicolour static at $4\times$. Tip gates and a runtime fallback to Smooth mitigated symptoms but did not beat Contour's geometric consistency.

c) Lesson.: For this medium, learn (or hard-code) *geometry* rather than continuous colour. Sparse teacher signal—only a fraction of a percent of HQ pixels differ from nearest under EPX/Scale2x—makes unweighted continuous losses dominated by flat regions.

VIII. DISCUSSION

Contour targets 45° stairs and shallow 2:1 wedges. Very irregular silhouettes and text may still benefit from Smooth's tip logic or Clarity's honest blocks. At modest integer scales (s=4 at 1080p), even a mathematically straight hard edge is a one-pixel staircase of *output* samples; that is rasterization, not the source-pixel zigzag Contour removes. Higher s (4K) makes continuous diagonals more visually obvious.

We intentionally omit Kopf spline optimisation: curved inked contours read as silky to many NES players. Hard half-planes keep the look closer to "drawn with exact colours" than "vectorized illustration."

IX. FPGA AND FUTURE WORK

Contour maps to a streaming pipeline: line buffers for a 3×3 window, YUV similarity comparators, four planar-diagonal units, and a few plane tests per output sample—no DRAM weight traffic. Clarity is already trivial on FPGA. Intended evaluation board: DE10-Nano for Clarity/Contour; larger devices only if a learned tip classifier returns.

Future work includes: tightening Contour on shallow non-45° hills and sprite silhouettes; an optional NES PAR (8:7) toggle for CRT-accurate width; a tip-only learned gate distilled from Smooth if Contour gaps remain; int8 / cycle-accurate RTL for the Contour datapath; and multi-game visual QA beyond Super Mario Bros.

X. CONCLUSION

RetroNNetScaler is a complete path from NES emulation to 4K presentation with explicit respect for discrete palettes. Clarity is the honest oracle; Smooth improves tips without cream; Contour makes long diagonals share one geometric edge instead of EPX's offset zigzags. A FPGA-sized neighbour CNN achieved zero palette drift but fringed until abandoned for Contour at runtime. The project's central lesson is: **discretise first**; for continuous edges without silk, make both sides of the stair agree on the same half-plane.

ACKNOWLEDGMENT

Training and validation dumps must use legally obtained ROMs; ROMs are not redistributed with the project.

REFERENCES

[1] K. Thomas, "Fast blit strategies: A Mac programmer's guide," *MacTech*, vol. 15, no. 6, 1999, documents Eric Johnston's EPX pixel-doubling rule (LucasArts, ca. 1992).

[2] A. Mazzoleni, "Scale2x," AdvanceMAME project documentation, 2001, archival primary source for Scale2x/Scale3x; functionally equivalent to EPX. [Online]. Available: https://www.scale2x.it

[3] P. M. Stasik and J. Balcerek, "Extensible implementation of reliable pixel art interpolation," *Foundations of Computing and Decision Sciences*, vol. 44, no. 2, pp. 213–239, 2019.

[4] J. W. Seo, J. W. Lee, J. H. Lee, J. B. Kim, and J.-W. Jung, "Structure-aware pixel art scaling via block size detection," *Applied Sciences*, vol. 16, no. 5, p. 2314, 2026.

[5] J. Kopf and D. Lischinski, "Depixelizing pixel art," *ACM Transactions on Graphics*, vol. 30, no. 4, pp. 99:1–99:8, 2011.

[6] F. Kreuzer, J. Kopf, and M. Wimmer, "Depixelizing pixel art in real-time," in *Proceedings of the 19th Symposium on Interactive 3D Graphics and Games*. ACM, 2015, p. 130, i3D 2015 poster / extended abstract.

[7] X. Wang, K. Yu, S. Wu, J. Gu, Y. Liu, C. Dong, Y. Qiao, and C. C. Loy, "ESRGAN: Enhanced super-resolution generative adversarial networks," in *Computer Vision — ECCV 2018 Workshops*. Springer, 2019, pp. 63–79.

[8] X. Wang, L. Xie, C. Dong, and Y. Shan, "Real-ESRGAN: Training real-world blind super-resolution with pure synthetic data," in *Proceedings of the IEEE/CVF International Conference on Computer Vision Workshops (ICCVW)*, 2021, pp. 1905–1914.

[9] J. Liang, J. Cao, G. Sun, K. Zhang, L. Van Gool, and R. Timofte, "SwinIR: Image restoration using swin transformer," in *Proceedings of the IEEE/CVF International Conference on Computer Vision Workshops (ICCVW)*, 2021, pp. 1833–1844.

[10] W. Shi, J. Caballero, F. Huszár, J. Totz, A. P. Aitken, R. Bishop, D. Rueckert, and Z. Wang, "Real-time single image and video super-resolution using an efficient sub-pixel convolutional neural network," in *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, 2016, pp. 1874–1883.